



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 10. Кейсы
(/teach/control/stream/view/id/931478155)

PErr10.2. Генератор логотипов с использованием Yandex-ART API. Часть 2 (new)

Предыдущий урок

(/pl/teach/control/lesson/view?id=346620939)

Следующий урок

(/pl/teach/control/lesson/view?id=346759562)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Что мы умеем

- ✓ Дорабатываем код для генерации логотипов с помощью Yandex Art API, используя seed для воспроизводимости и стилистические пресеты.
- ✓ Создаем веб-приложения на Flask с HTML-шаблонами, JavaScript для интерактивности и обработкой POST/GET-запросов.
- ✓ Развертываем Flask-приложения на VPS-сервере, настраиваем виртуальное окружение, .env-файлы и фоновый запуск через screen.

Результат дня

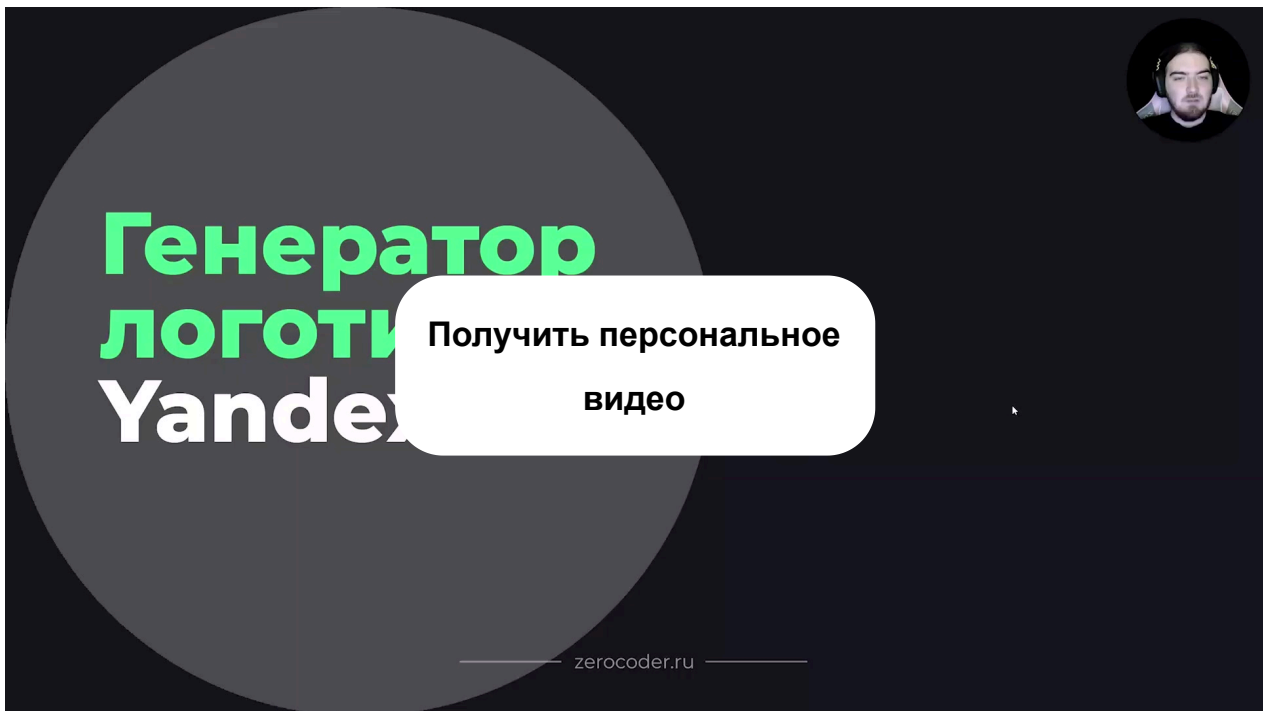
Узнаешь, как доработать генератор логотипов на Yandex Art API, упаковать его в полноценный веб-сайт на Flask и развернуть на VPS-сервере для доступа через браузер без кода.

▶ Время чтения и просмотра: ~ 1,5 часа

🕒 Время выполнения задания: ~ 55 минут

Теория на сегодня

1. Цели урока и возможности Yandex-ART



Сегодня мы продолжаем курс по промпт-инжинирингу и дорабатываем генератор логотипов, начатый на прошлом уроке. Возьмем готовый скрипт, улучшим его и упакуем решение в полноценный сайт, где пользователь сможет сгенерировать и скачать изображение.

Действия

Сегодня мы:

1. Повторим, какие задачи можно решать с помощью Yandex ART.
2. Вспомним способ взаимодействия с сервером через API.
3. Допишем код и реализуем сайт-генератор логотипов.

▼ Цели урока

- ✓ Цель текущего урока доработать сервис для работы с ним через сайт.

1. Повторим возможности использования сервиса генерации изображений, разберем для каких задач он применяется и какие сценарии работы с ним возможны.
2. Повторим схема взаимодействия с сервером через **API**: кратко освежим теорию, обсудим принципы работы веб-сервера, **HTTP**-методы запросов и общую логику взаимодействия клиент–сервер.
3. Отдельный фокус занятия — доработка кода и реализация веб-сайта для генерации логотипов.
4. Финальной целью является запуск полноценного сайта-генератора логотипов. Для этого рассмотрим фреймворк **Flask**, его назначение и базовые принципы создания веб-приложений.
5. Развернем сайт на сервере, чтобы любой пользователь мог зайти на него через браузер и воспользоваться функциональностью сервиса без необходимости работы с кодом.

Генерация логотипов уже реализована на базовом уровне: при передаче текстового запроса на выходе получается изображение. Однако формат взаимодействия через код неудобен для конечного пользователя и плохо подходит для практического использования. Поэтому упакуем сервис в понятный и удобный пользовательский интерфейс. Через сайт пользователи смогут генерировать логотипы, обновлять результат и при необходимости корректировать генерацию.

Сайт будет работать с сервисом генерации изображений через **API**. Реализуем простой интерфейс с полем для ввода текстового запроса и возможностью выбора предустановленных шаблонов. На выходе

пользователь будет получать изображение логотипа, которое при необходимости можно дорабатывать с помощью дополнительных запросов.

▼ Возможности Yandex-ART

Рассмотрим основные возможности сервиса генерации изображений и то, как они используются в рамках проекта.

1. Первая и ключевая функция — преобразование текстового запроса в изображение.

✓ **Текст в изображение** — это стандартная функция генеративных моделей генерации фото из текстового описания(промпта).

Пользователь вводит текстовый запрос, на основе которого формируется готовое изображение, пригодное для дальнейшего использования.

2. Вторая важная возможность — гибкая настройка стиля результата.

✓ **Гибкие стили** — возможность изменять финальный вид получаемого изображения, изменяя промпт.

Финальный внешний вид изображения можно изменять за счет корректировки текстового запроса: добавлять детали, уточнять стиль и тем самым управлять результатом генерации.

3. Третья функция — **API** доступ.

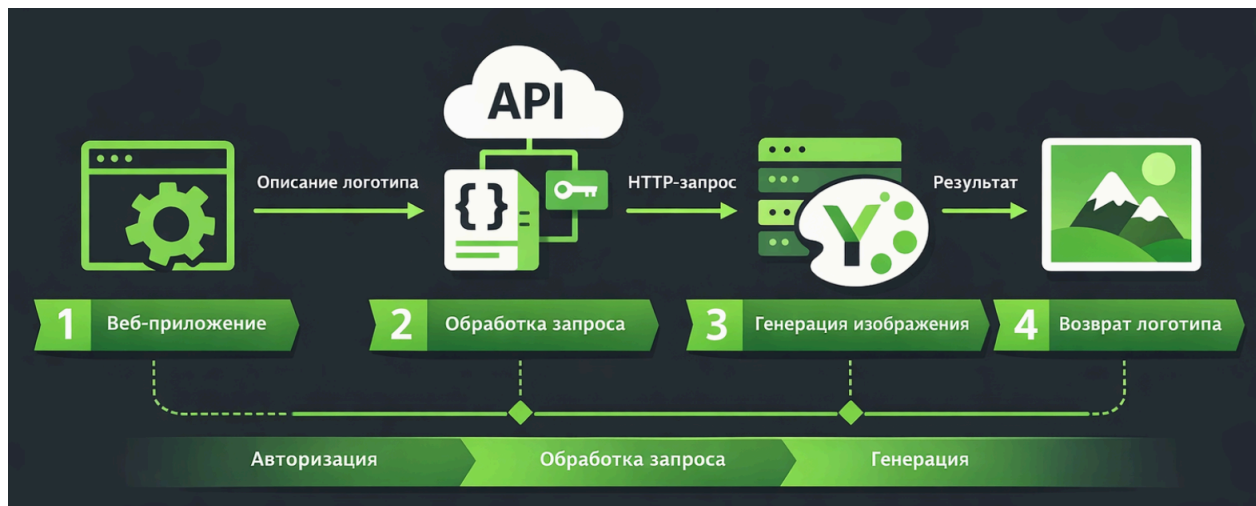
✓ **API доступ** — возможность работать с моделью по удобной **API** (набор удобных правил, инструментов и протоколов).

Вся работа с моделью осуществляется через **API**.

✓ **API** представляет собой набор правил и инструментов, позволяющих организовать взаимодействие между веб-сайтом и сервером сервиса генерации изображений.

Через **API** запросы с сайта передаются на сервер, где выполняется генерация, после чего результат возвращается обратно в приложение.

Рассмотрим, как работает API-запрос.

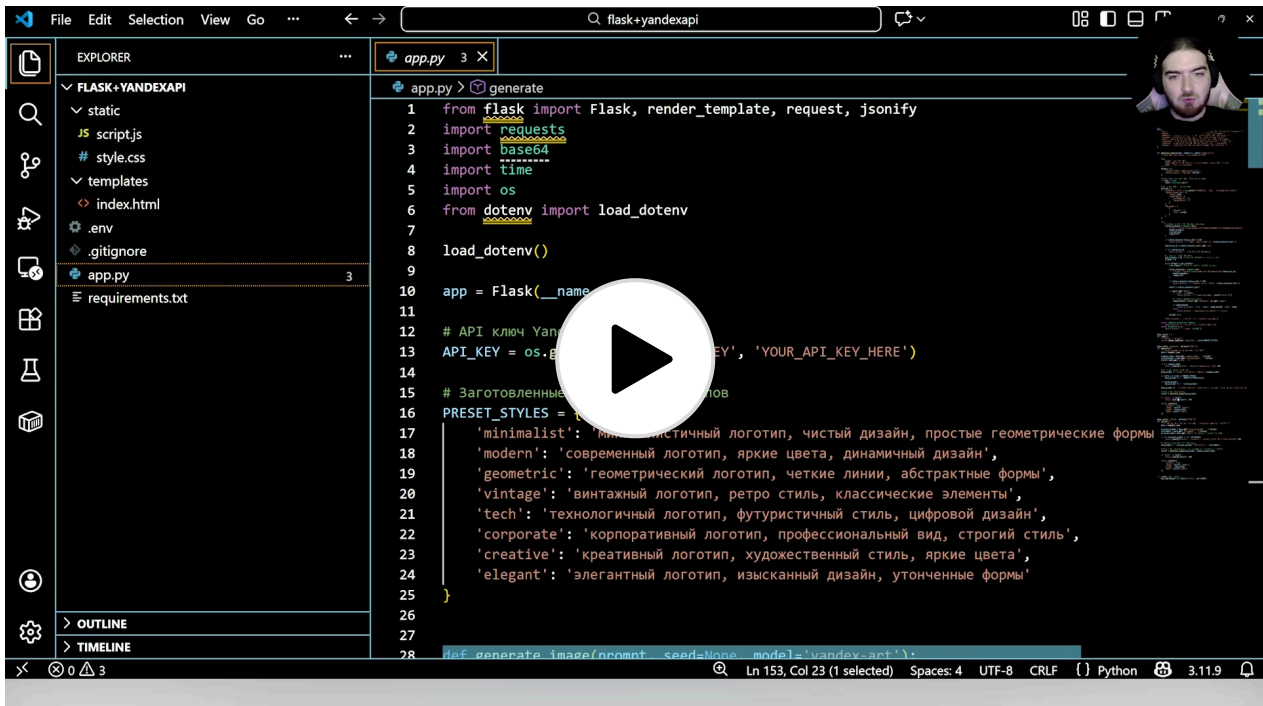


- Существует точка входа — источник запроса. Это может быть веб-приложение, **Telegram-бот**, сайт или скрипт, запущенный в терминале. Формат источника не имеет принципиального значения: важно лишь то, что именно из него инициируется запрос.
- Далее сформированный запрос передается через **API** на сервер сервиса. Внутри запроса указываются все необходимые параметры: текстовый запрос, данные для доступа, служебная информация и прочие обязательные элементы.
- После получения запроса сервер обрабатывает его, формирует результат и возвращает ответ обратно в то приложение, из которого запрос был отправлен.

✓ Базовая схема выглядит максимально просто: приложение отправляет запрос → сервер обрабатывает его → приложение получает ответ.

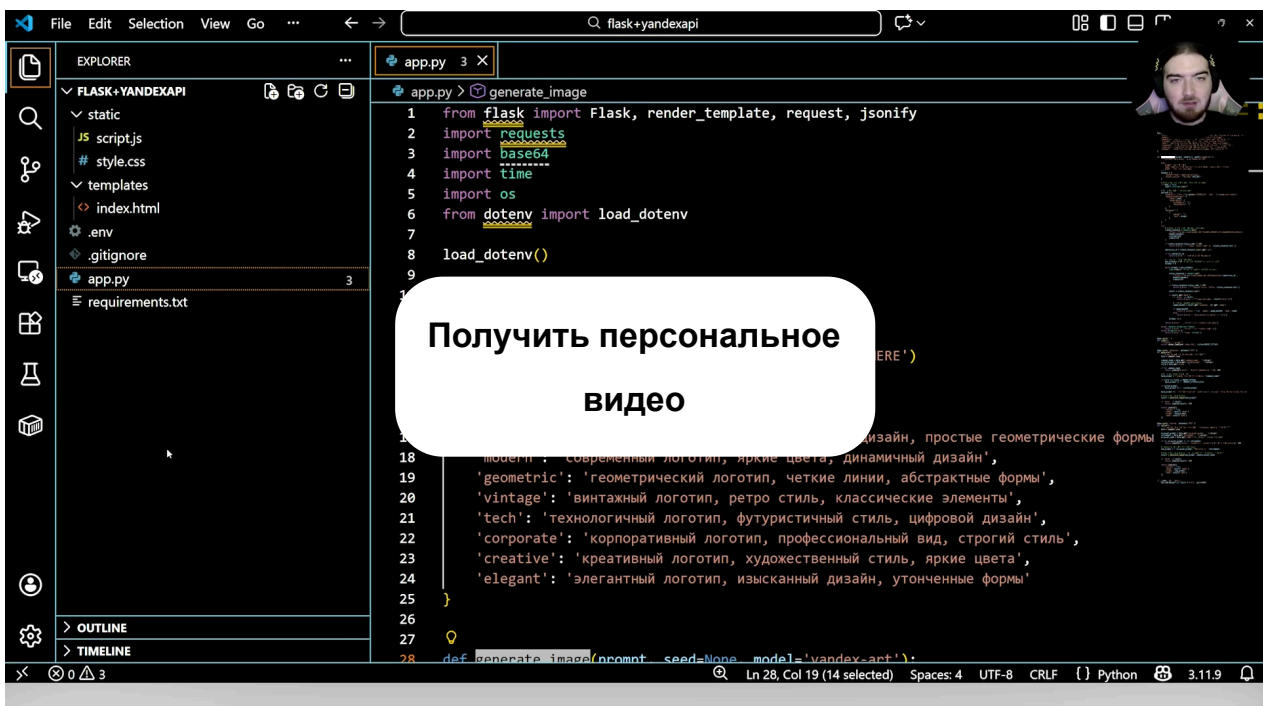
Это универсальный и удобный способ взаимодействия с внешними сервисами. API используется повсеместно, и практически любое подключение к сторонним ресурсам сводится именно к работе через API.

2. Логика выполнения кода: каким образом формируются и отправляются запросы

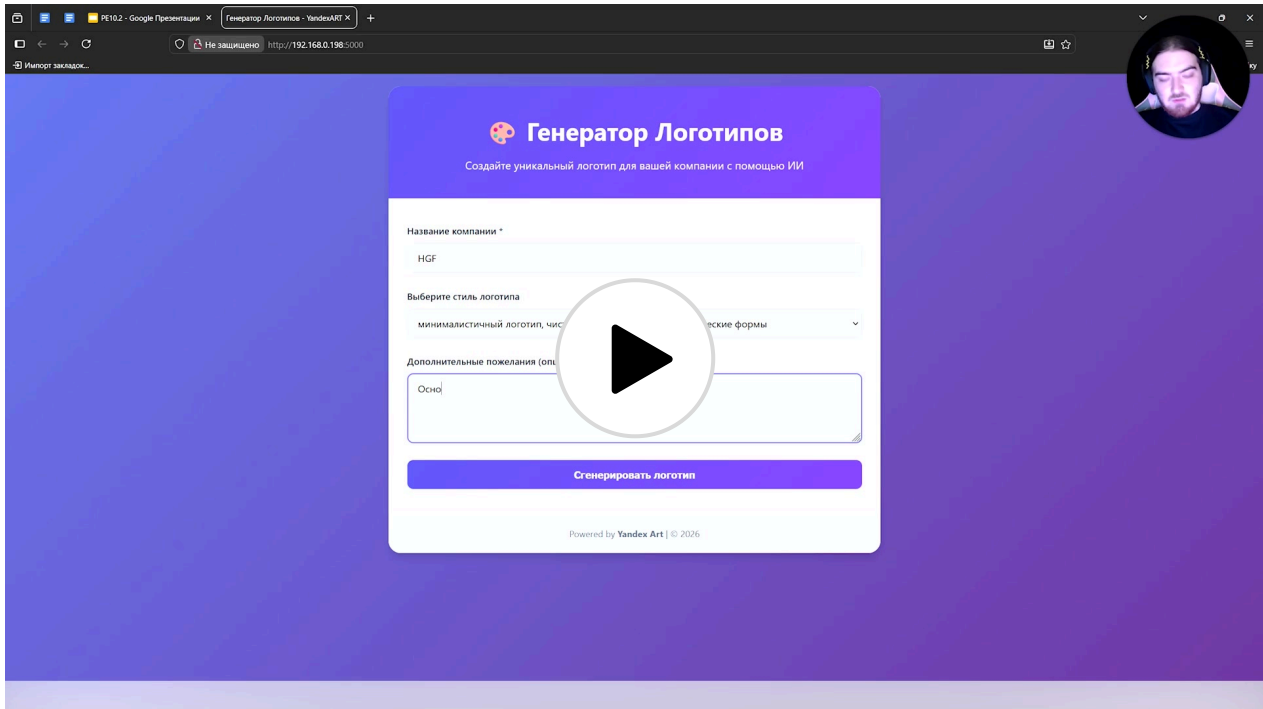


► Логика выполнения кода

3. Создание веб-приложений с помощью Flask

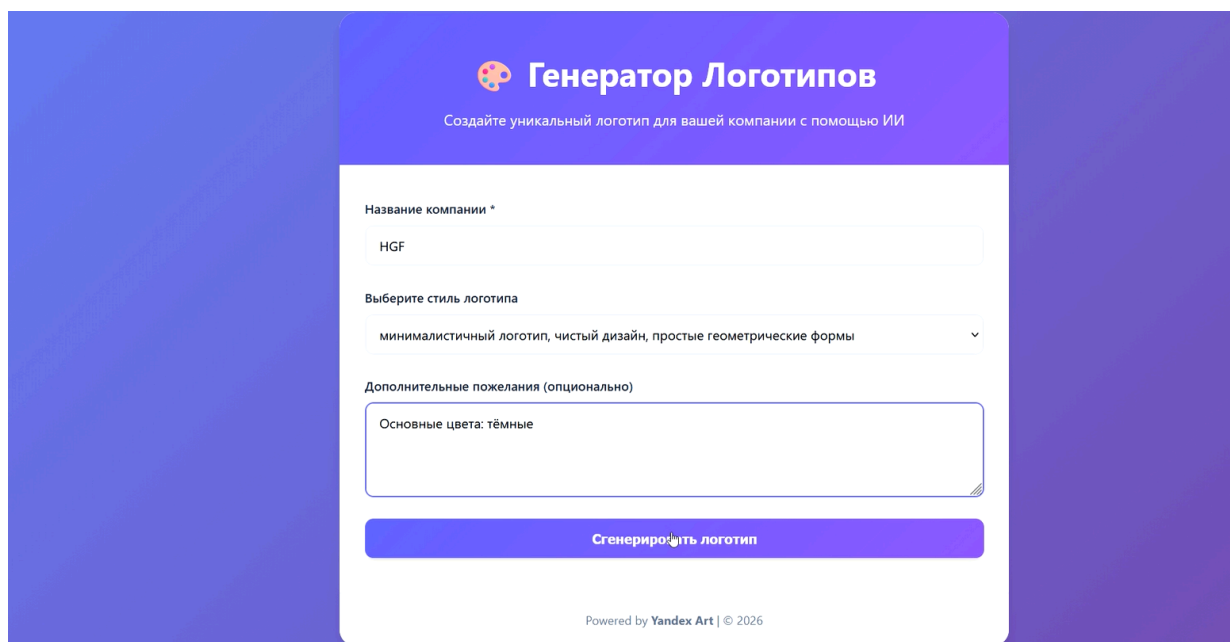


4. Тестирование созданного веб-приложения



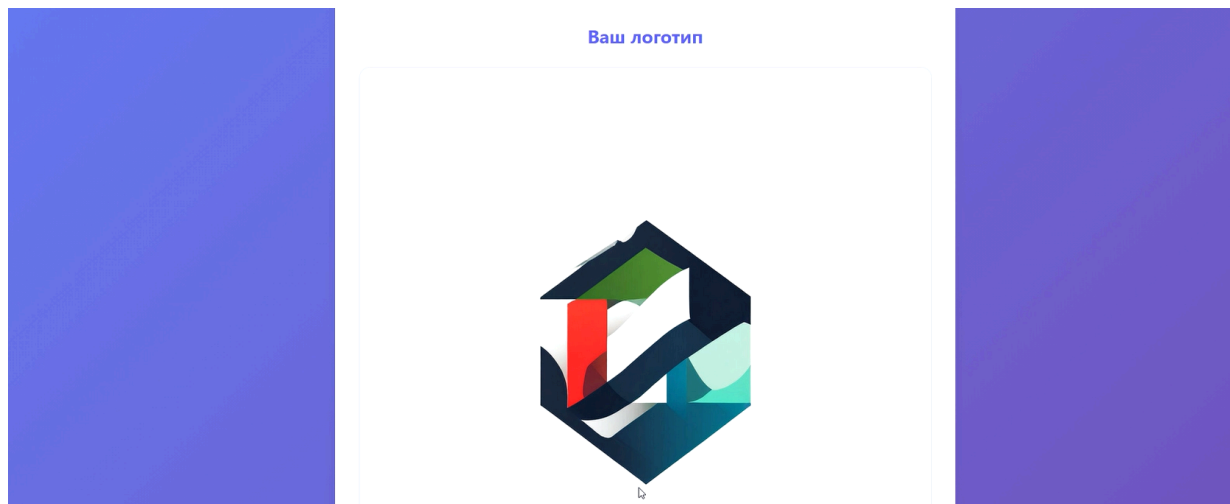
▼ Тестирование сайта

- Обновляем страницу в браузере → проверяем, что сайт корректно загрузился и готов к работе.
- Вводим данные для генерации логотипа: название компании: например, HGF → выбираем стиль логотипа → указываем основные цвета (допустим, темные) → нажимаем кнопку Сгенерировать → запрос отправляется на сервер → в логах видим: POST /generate 200 — ответ успешно получен.



Действия

- Через несколько секунд появляется сгенерированное изображение. Результат есть, но качество не самое удачное — модель сгенерировала логотип не так, как хотелось бы.



- Создаем новый логотип с другим названием: вводим более корректное имя, например: Daniil → указываем основные цвета: черный и серый.

- Делаем небольшую правку в коде: в коде есть условие, что логотип должен быть на белом или прозрачном фоне. Временно комментируем эту часть, сохраняем файл. **Flask** автоматически перезапускается.

```
149 #base_prompt += ' Логотип должен быть на прозрачном или белом фоне, профессиональны
```

- Обновляем страницу и запускаем генерацию заново → снова видим 200 — запрос дошел, генерация началась → цвета применяются, но результат все еще не идеален.

Ваш логотип



- Уточняем промпт вручную, прописываем:



Нужно, чтобы на логотипе было написано "Daniil".

- Ждем повторную генерацию. Генерация с использованием seed занимает немного больше времени.

Использованный промпт:

Создай логотип для компании "Daniil". минималистичный логотип, чистый дизайн, простые геометрические формы. Основные цвета: тёмные, чёрный и серый.

Хотите дополнить логотип?

Дополнение сохранит базовую структуру логотипа и добавит ваши изменения

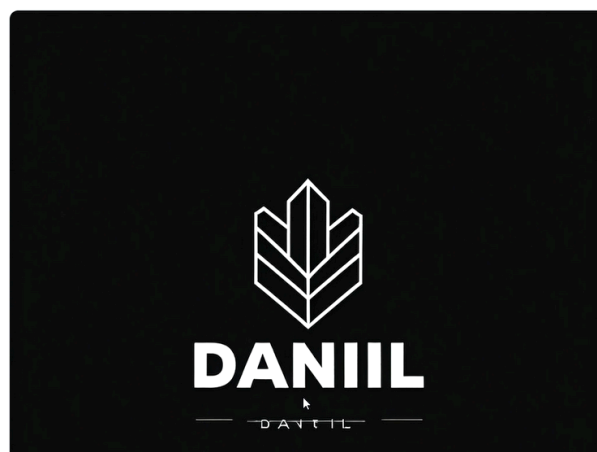
Нужно чтобы на логотипе было написано "DANIIL"

Дополнить логотип

Создать новый логотип

- Получаем обновленный результат: стиль остался похожим на предыдущий вариант, но теперь появился нужный текст. Это значит, что изображение успешно доработано, не генерируя его с нуля.

Ваш логотип



Действия

- При желании можно сменить стиль, например, на винтажный, и снова отправить запрос.



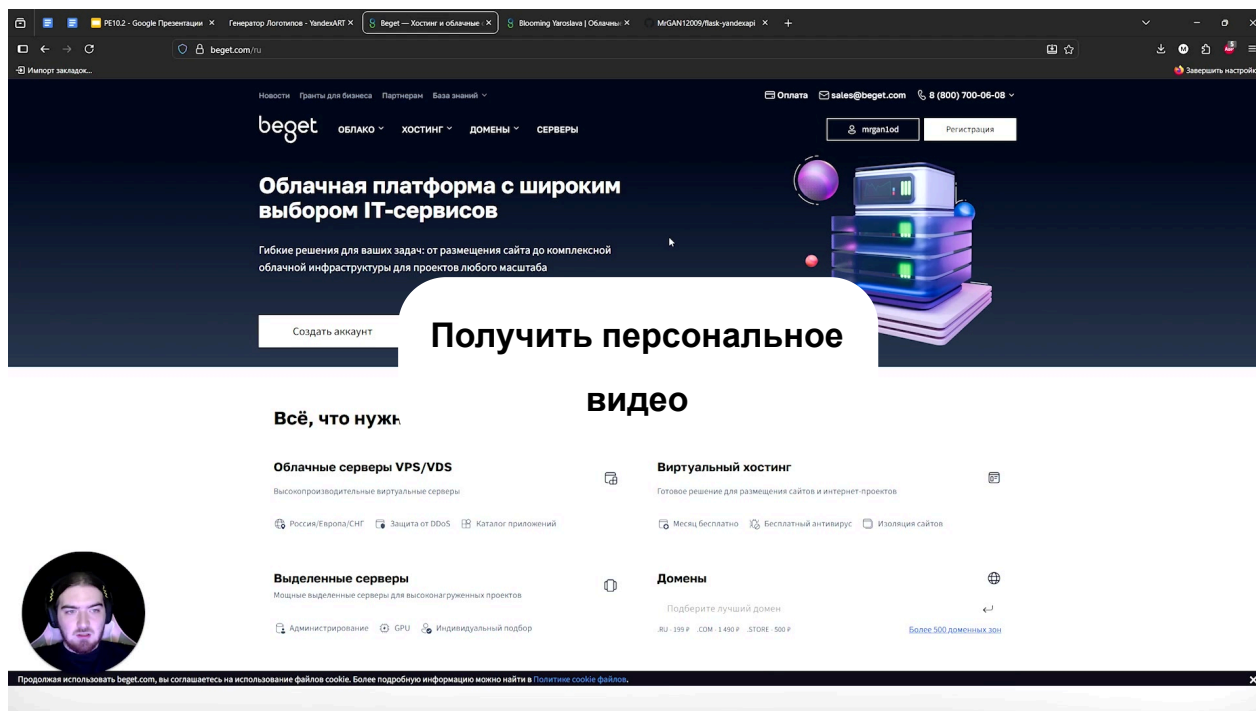
- В **Terminal** отображается последовательность действий: первый `POST /generate` — начальная генерация → `refine` — доработка изображения → повторный `generate` — финальный вариант.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS python + v [ ] [ ] ... [ ]
```

```
* Detected change in 'C:\\Users\\daniil\\Desktop\\flask+yandexapi\\app.py', reloading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 976-715-519
192.168.0.198 - - [28/Jan/2026 20:56:16] "GET / HTTP/1.1" 200 -
192.168.0.198 - - [28/Jan/2026 20:56:16] "GET /static/style.css HTTP/1.1" 304 -
192.168.0.198 - - [28/Jan/2026 20:56:16] "GET /static/script.js HTTP/1.1" 304 -
192.168.0.198 - - [28/Jan/2026 20:56:24] "POST /generate HTTP/1.1" 200 -
192.168.0.198 - - [28/Jan/2026 20:57:03] "POST /refine HTTP/1.1" 200 -
192.168.0.198 - - [28/Jan/2026 20:57:36] "POST /generate HTTP/1.1" 200 -
```

⚠ Сайт работает корректно, логика понятная, все действия происходят без перезагрузки страницы. Пользователь вводит данные, нажимает кнопку, ждет результат и при необходимости дорабатывает логотип — быстро и удобно.

5. Запуск сайта на VPS-сервере



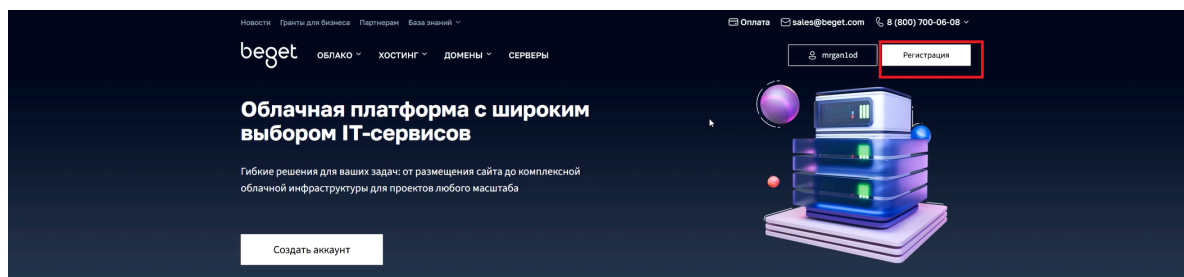
▼ Запуск сайта на сервере

Разберем, как запустить сайт на сервере. Для этого в первую очередь понадобится **VPS** — виртуальный сервер.

Существует много провайдеров, но для примера можно взять один из самых простых и недорогих вариантов. Минимальная конфигурация такого сервера стоит около нескольких рублей в день и подходит для размещения небольшого сайта или учебного проекта.

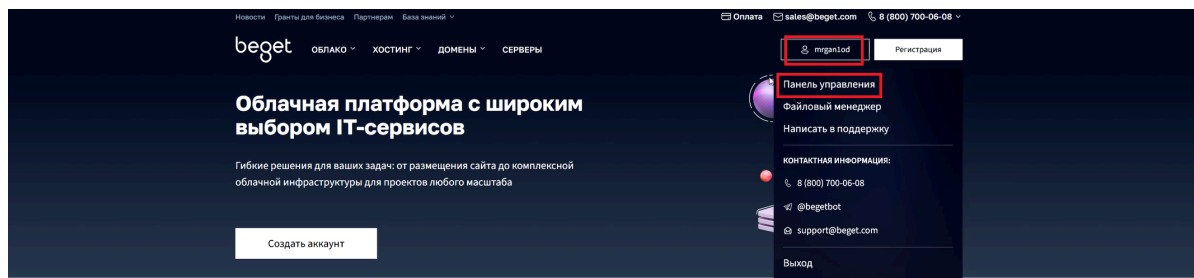
1. Регистрация у VPS-провайдера:

- Переходим на сервер **Beget**: если аккаунта еще нет нажимаем **Регистрация**.

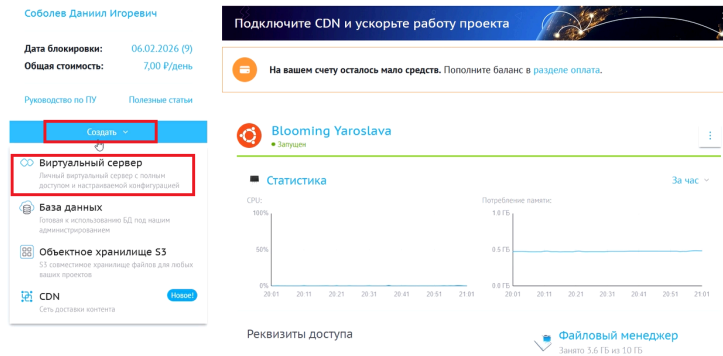


- После регистрации выполняем вход в **Панель управления сервера**.

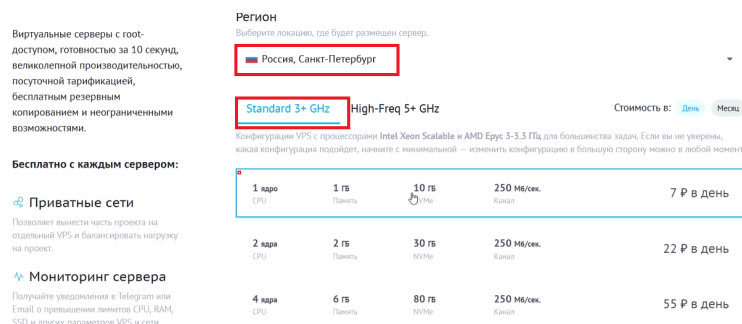
Действия



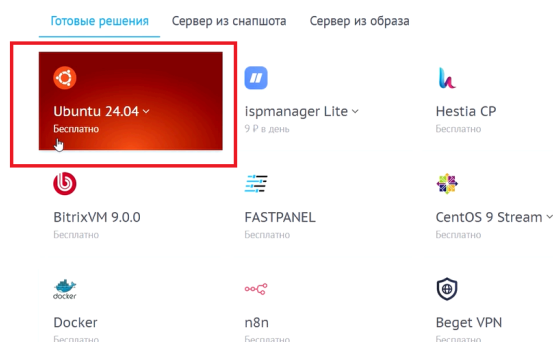
- В панели управления выбираем пункт создания нового сервера: нажимаем Создать → тип сервера — Виртуальный сервер .



- Для небольшого проекта достаточно минимального тарифа. Этой конфигурации хватает для запуска **Flask**-приложения и обработки запросов. Можно выбрать один из доступных регионов (например, Москва или Санкт-Петербург). Другие регионы, как правило, стоят дороже, но принцип работы одинаковый.

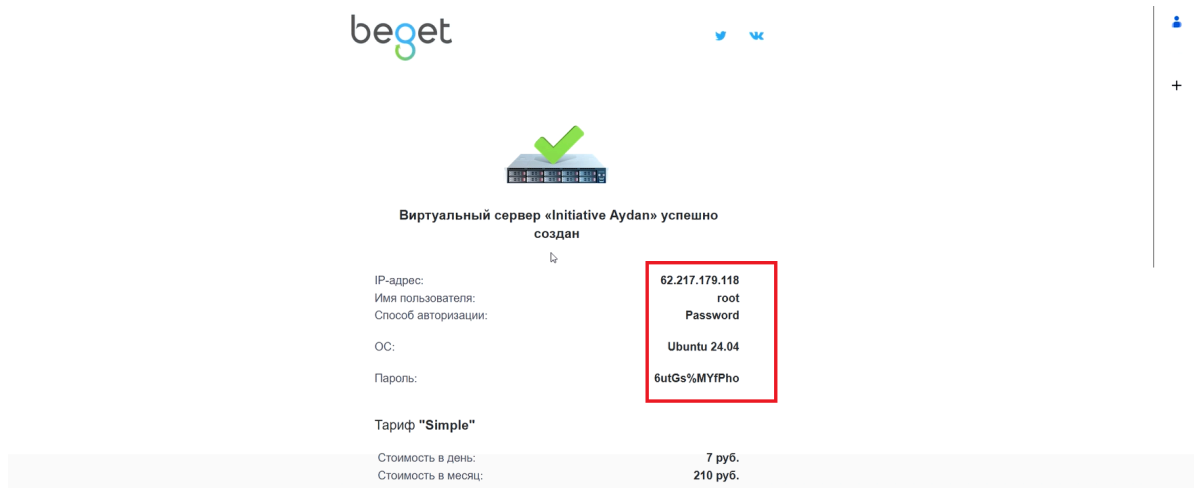


- В качестве операционной системы выбираем стандартный **Linux**, чаще всего **Ubuntu** . Это универсальный вариант, подходящий для большинства задач. Дополнительные решения вроде **Docker**, **CMS** или автоматизаций при необходимости можно установить вручную позже. После подтверждения параметров запускается процесс создания **VPS**. Через некоторое время сервер становится активным.



Действия

- Лучше заранее разместить код проекта в облачном репозитории, например в **Git**-репозитории. Это упрощает загрузку кода на сервер и дальнейшую работу. Из среды разработки можно опубликовать проект в репозиторий, следуя стандартным шагам. Цель — иметь удаленное хранилище кода, откуда сервер сможет быстро получить проект.
- После создания **VPS** на почту приходит письмо с доступами: **IP**-адрес сервера; имя пользователя (обычно **root**); пароль.

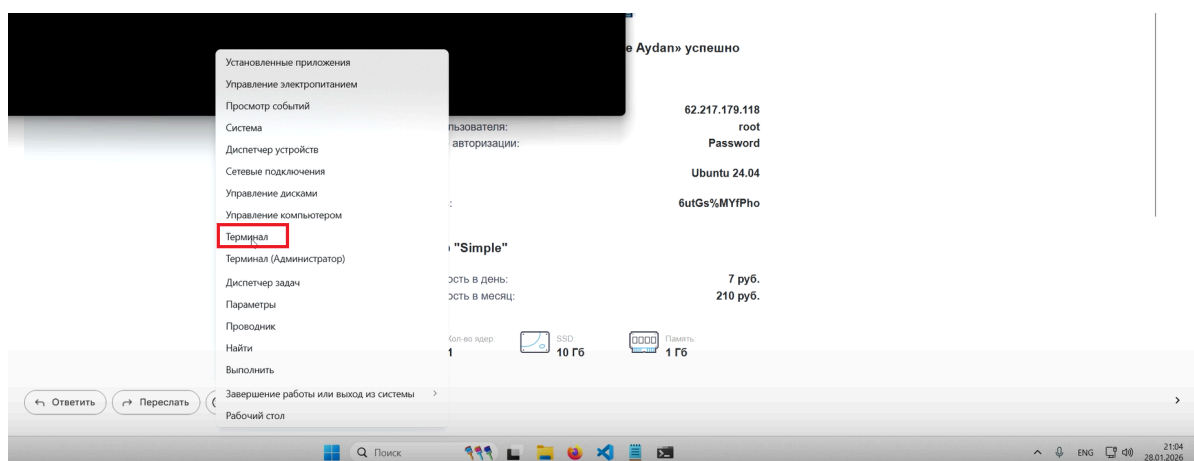


Эти данные используются для подключения к серверу.

Независимо от выбранного хостинг-провайдера, принцип всегда одинаковый: после создания **VPS** предоставляется **root**-доступ с **IP**-адресом и паролем.

2. Подключение к серверу и настройка окружения:

- Переходим в **Terminal** → на **Windows** можно нажать правой кнопкой мыши и выбрать пункт Терминал .



- Для подключения используем команда **ssh** → указываем имя пользователя и IP-адрес сервера в формате: **ssh пользователь@ip_адрес** → при первом подключении система спросит подтверждение доверия к серверу → подтверждаем → терминал запросит пароль → пароль вставляем обычной вставкой (в терминале он не отображается).

Действия

```
PS C:\Users\daniil> ssh root@62.217.179.118
The authenticity of host '62.217.179.118 (62.217.179.118)' can't be established.
ED25519 key fingerprint is SHA256:nt6urZXX1Lf1ItWV9peeVRP94J8/y9HyeM1EDto0LK8.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '62.217.179.118' (ED25519) to the list of known hosts.
root@62.217.179.118's password:
```

- После успешного входа меняется командная строка (prompt). Это означает, что работа ведется уже не на локальном компьютере, а на удаленном сервере.
- Копируем проект на сервер, используем команду `git clone` → вставляем ссылку на репозиторий. После выполнения команды начинается процесс клонирования проекта.

```
root@yypjwcbejk:~# git clone https://github.com/MrGAN12009/flask-yandexapi
Cloning into 'flask-yandexapi'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 13 (delta 1), reused 13 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 7.89 KiB | 3.95 MiB/s, done.
Resolving deltas: 100% (1/1), done.
root@yypjwcbejk:~#
```

- Команда `ls` выводит список файлов и папок в текущей директории. В списке появляется папка проекта с названием, соответствующим репозиторию. Для перехода используем команду: `cd <название_папки>`. После этого в командной строке отображается текущая директория проекта.

```
root@yypjwcbejk:~# git clone https://github.com/MrGAN12009/flask-yandexapi
Cloning into 'flask-yandexapi'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 13 (delta 1), reused 13 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 7.89 KiB | 3.95 MiB/s, done.
Resolving deltas: 100% (1/1), done.
root@yypjwcbejk:~# ls
flask-yandexapi
root@yypjwcbejk:~# cd flask-yandexapi
root@yypjwcbejk:~/flask-yandexapi#
```

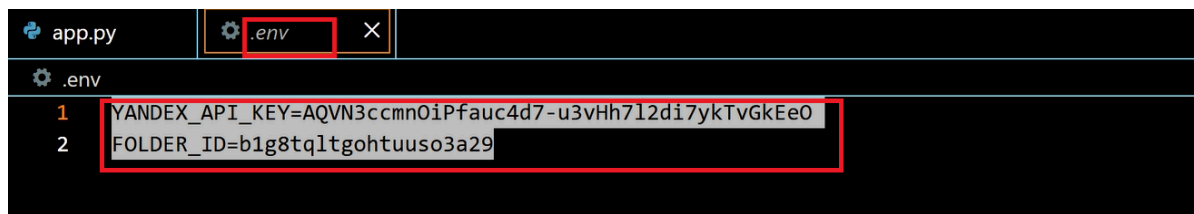
- На **Linux**-серверах **Python 3**, как правило, уже установлен по умолчанию. Поэтому создаем виртуальное окружение стандартной командой: `python3 -m venv venv`. После выполнения команды в директории проекта появляется папка виртуального окружения.
- Активируем виртуальное окружение, используя команду: `source venv/bin/activate`. После активации виртуального окружения командная строка снова изменяется — это признак, что окружение активно.
- Устанавливаем зависимости проекта. Все внешние библиотеки устанавливаем из файла зависимостей: `pip install -r requirements.txt`. После выполнения команды начинается загрузка и установка необходимых пакетов.

Действия

- Создаем файл с переменными окружения и сразу открываем для редактирования, например с помощью текстового редактора: `nano .env`. В этом файле будут храниться ключи и настройки, необходимые для работы приложения.

3. Запуск приложения и проверка работы сайта:

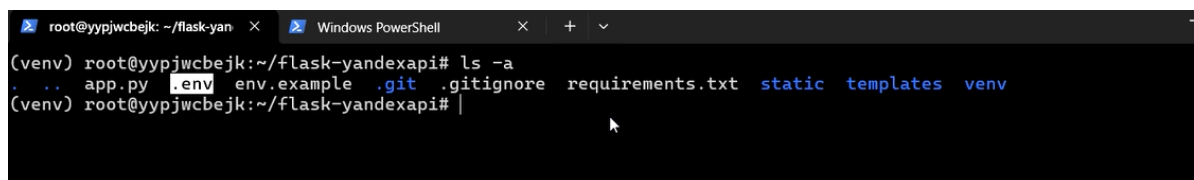
- Возвращаемся в редактор кода → копируем содержимое примера файла с переменными окружения, вставляем его в созданный на сервере файл `.env`.



The screenshot shows a code editor with two tabs: `app.py` and `.env`. The `.env` tab is active and contains the following content:

```
1 YANDEX_API_KEY=AQVN3ccmn0iPfauc4d7-u3vHh7l2di7ykTvGkEe0
2 FOLDER_ID=b1g8tqltgohtuuso3a29
```

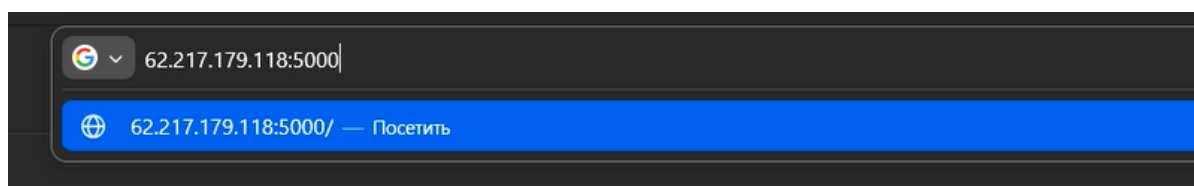
- Сохраняем изменения и выходим из редактора `Ctrl + X` → `Y` → `Enter` → выходим из редактора → вводим команду просмотра файлов. В списке должен появиться файл `.env`, что подтверждает его успешное создание.



The screenshot shows a terminal window with the following command and output:

```
(venv) root@yypjwcbejk: ~/flask-yandexapi# ls -a
.  ..  app.py  .env  env.example  .git  .gitignore  requirements.txt  static  templates  venv
```

- Запускаем сервер командой: `python app.py`. Приложение стартует и начинает слушать указанный порт. В конце запуска приложения указано, на каком порту работает сервер. В данном случае используется порт `5000`.
- Открываем браузер и в адресной строке вводим: `http://IP_АДРЕС_СЕРВЕРА:5000`. После этого сайт становится доступен из любой точки мира.



- На странице можно выбрать стиль логотипа, указать описание и запустить генерацию. После отправки запроса начинается процесс генерации изображения.

- В терминале сервера отобразятся логи обработки запросов. По ним видно, что запрос на генерацию принят и успешно обработан. В результате логотип успешно генерируется и отображается в браузере. Возможны визуальные артефакты — это особенность работы модели генерации изображений и не связано с настройкой сервера.

⚠ Сайт полностью запущен на удаленном сервере и доступен через интернет. В таком виде проект уже можно демонстрировать заказчику или использовать для презентации решения.

4. Фоновый запуск приложения:

Если запустить приложение обычной командой и закрыть терминал, сервер останавливается. В браузере страница перестает загружаться, так как процесс **Python** завершен. Для решения данной проблемы необходимо осуществить запуск в фоне через `screen` :

- Подключаемся к серверу по **SSH** → переходим в директорию с проектом командой смены директории → вводим команду: `screen` . Создается виртуальная сессия терминала, которая продолжает работать даже после закрытия основного окна.

```
root@yypjwcbejk: ~/flask-yan  x  +  v
root@yypjwcbejk:~# cd flask-yandexapi/
root@yypjwcbejk:~/flask-yandexapi# screen
```

- Внутри `screen` активируем ранее созданное виртуальное окружение: `source venv/bin/activate` . Все зависимости уже установлены, повторно ничего ставить не нужно. Запускаем сервер: `python app.py` . Приложение стартует и начинает принимать запросы.

Действия

```
root@yypjwcbejk: ~/flask-yan  x + v
root@yypjwcbejk:~/flask-yandexapi# source venv/bin/activate
(venv) root@yypjwcbejk:~/flask-yandexapi# python app.py |
```

- Открываем сайт в браузере по **IP**-адресу и порту → пробуем сгенерировать логотип → запрос обрабатывается, результат возвращается → закрываем окно терминала или **SSH**-сессию. При этом приложение продолжает работать, так как запущено внутри **screen** .

```
root@yypjwcbejk: ~/flask-yan  x + v
root@yypjwcbejk:~/flask-yandexapi# source venv/bin/activate
(venv) root@yypjwcbejk:~/flask-yandexapi# python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://62.217.179.118:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 437-676-348
178.155.127.84 - - [28/Jan/2026 18:12:13] "GET / HTTP/1.1" 200 -
178.155.127.84 - - [28/Jan/2026 18:12:14] "GET /static/script.js HTTP/1.1" 304 -
178.155.127.84 - - [28/Jan/2026 18:12:14] "GET /static/style.css HTTP/1.1" 304 -
178.155.127.84 - - [28/Jan/2026 18:12:23] "POST /generate HTTP/1.1" 200 -
```

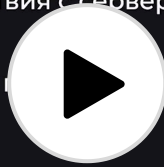
- Обновляем страницу в браузере. Сайт по-прежнему доступен, генерация работает корректно.

Приложение запущено на сервере в фоновом режиме, не зависит от открытого терминала и стабильно работает. В результате получен полностью функционирующий сайт для генерации логотипов, доступный из интернета и готовый к демонстрации пользователям или заказчикам.

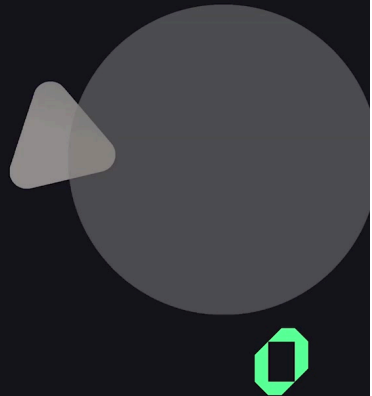
6. Итоги урока

Итоги урока

1. Повторили что мы можем сделать используя Yandex-ART
2. Повторили способ взаимодействия с сервером через API
3. Дописали код, реализовав сайт генератор логотипов



zerocoder.ru



▼ Результаты урока

1. Повторили что мы можем сделать используя Yandex-ART.
2. Повторили способ взаимодействия с сервером через API.
3. Дописали код, реализовав сайт генератор логотипов.

Дополнительные материалы

▼ Код проекта эксперта

Код проекта эксперта на GitHub

(<https://github.com/MrGAN12009/flask-yandexapi>)

► Ссылки на программы и сайты

Домашнее задание

- Доработай веб-приложение для генерации логотипов.
- В качестве домашнего задания пришли готовый сгенерированный логотип и скриншоты веб-приложения.

Действия

На следующем занятии

На следующем занятии мы создадим свой собственный генератор видео: разберём инструменты и API для автоматизации генерации, научимся интегрировать промпты для описания сцен, стилей и поймём, как масштабировать это для реальных проектов в промпт-инжиниринге.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.2. Генератор логотипов с использованием Yandex-ART API. Часть 2

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

- Доработай веб-приложение для генерации логотипов.
Действия

- В качестве домашнего задания пришли готовый сгенерированный логотип и скриншоты веб-приложения.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=346620939>)

Следующий урок

Действия